



Tensors and Einsum

Max Koch, Tamino Steinert

Tensors

Tensors

- Tensors are the basic data structure in many ML frameworks, e.g., PyTorch or Tensorflow
- Tensors are multidimensional arrays and can represent, e.g., scalars, vectors and matrices
- Tensor contractions can perform linear algebra operations, e.g., dot product, outer product and matrix multiplication
- Tensor contractions can be expressed with *einsum*

```
1 import torch
2
3 # 0D tensor - scalar
4 s = torch.tensor(1)
5
6 # 1D tensor - vector
7 v = torch.tensor([1, 2])
8
9 # 2D tensor - matrix
10 m = torch.tensor([[1, 2],
11                  [3, 4]])
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
```

Tensors

- Tensors are the basic data structure in many ML frameworks, e.g., PyTorch or Tensorflow
- Tensors are multidimensional arrays and can represent, e.g., scalars, vectors and matrices
- Tensor contractions can perform linear algebra operations, e.g., dot product, outer product and matrix multiplication
- Tensor contractions can be expressed with *einsum*

```
1 import torch
2
3 # 0D tensor - scalar
4 s = torch.tensor(1)
5
6 # 1D tensor - vector
7 v = torch.tensor([1, 2])
8
9 # 2D tensor - matrix
10 m = torch.tensor([[1, 2],
11                  [3, 4]])
12
13 # 3D tensor - "cube"
14 c = torch.tensor([[[1, 2],
15                  [3, 4]],
16                  [[5, 6],
17                  [7, 8]]])
18
19 # ...
20
21
22
23
24
25
26
27
28
29
30
```

Tensors

- Tensors are the basic data structure in many ML frameworks, e.g., PyTorch or Tensorflow
- Tensors are multidimensional arrays and can represent, e.g., scalars, vectors and matrices
- Tensor contractions can perform linear algebra operations, e.g., dot product, outer product and matrix multiplication
- Tensor contractions can be expressed with *einsum*

```
1 import torch
2
3 # 0D tensor - scalar
4 s = torch.tensor(1)
5
6 # 1D tensor - vector
7 v = torch.tensor([1, 2])
8
9 # 2D tensor - matrix
10 m = torch.tensor([[1, 2],
11                   [3, 4]])
12
13 # 3D tensor - "cube"
14 c = torch.tensor([[[1, 2],
15                   [3, 4]],
16                   [[5, 6],
17                   [7, 8]]])
18
19 # ...
20
21 # dot product
22 s2 = torch.dot(v, v)
23 # vector-matrix multiplication
24 v2 = torch.matmul(v, m)
25 # matrix-matrix multiplication
26 m2 = torch.matmul(m, m)
27
28
29
30
```

Tensors

- Tensors are the basic data structure in many ML frameworks, e.g., PyTorch or Tensorflow
- Tensors are multidimensional arrays and can represent, e.g., scalars, vectors and matrices
- Tensor contractions can perform linear algebra operations, e.g., dot product, outer product and matrix multiplication
- Tensor contractions can be expressed with *einsum*

```
1 import torch
2
3 # 0D tensor - scalar
4 s = torch.tensor(1)
5
6 # 1D tensor - vector
7 v = torch.tensor([1, 2])
8
9 # 2D tensor - matrix
10 m = torch.tensor([[1, 2],
11                  [3, 4]])
12
13 # 3D tensor - "cube"
14 c = torch.tensor([[[1, 2],
15                  [3, 4]],
16                  [[5, 6],
17                  [7, 8]]])
18
19 # ...
20
21 # dot product
22 s2 = torch.dot(v, v)
23 # vector-matrix multiplication
24 v2 = torch.matmul(v, m)
25 # matrix-matrix multiplication
26 m2 = torch.matmul(m, m)
27
28 s3 = torch.einsum("k, k ->", v, v)
29 v3 = torch.einsum("k, kn -> n", v, m)
30 m3 = torch.einsum("mk, kn -> mn", m, m)
```

Tensors

- Tensors are the basic data structure in many ML frameworks, e.g., PyTorch or Tensorflow
- Tensors are multidimensional arrays and can represent, e.g., scalars, vectors and matrices
- Tensor contractions can perform linear algebra operations, e.g., dot product, outer product and matrix multiplication
- Tensor contractions can be expressed with *einsum*

```
1 import torch
2
3 # 0D tensor - scalar
4 s = torch.tensor(1)
5
6 # 1D tensor - vector
7 v = torch.tensor([1, 2])
8
9 # 2D tensor - matrix
10 m = torch.tensor([[1, 2],
11                  [3, 4]])
12
13 # 3D tensor - "cube"
14 c = torch.tensor([[[1, 2],
15                  [3, 4]],
16                  [[5, 6],
17                  [7, 8]]])
18
19 # ...
20
21 # dot product
22 s2 = torch.dot(v, v)
23 # vector-matrix multiplication
24 v2 = torch.matmul(v, m)
25 # matrix-matrix multiplication
26 m2 = torch.matmul(m, m)
27
28 s3 = torch.einsum("k, k ->", v, v)
29 v3 = torch.einsum("k, kn -> n", v, m)
30 m3 = torch.einsum("mk, kn -> mn", m, m)
```

Mapping Tensors to Memory

- Tensors are multidimensional arrays
- Elements can be accessed through indices
- Memory / address space is linear — no concept of higher dimensions
- The mapping between memory and tensor indices is determined by its *strides*
- The stride of a dimension specifies how many elements to jump to access the next element along that dimension
- We assume general row-major format in this course — dimensions have descending strides, i.e., the last dimension has stride 1

```
1 import torch
2 # 3D tensor where each dimension size is two
3 t = torch.tensor([[[1, 2],
4                   [3, 4]],
5                  [[5, 6],
6                   [7, 8]]], dtype=torch.int8)
7
8 print(t.shape) # torch.Size([2, 2, 2])
9
10 print(t[0, 0, 0]) # tensor(1)
11 print(t[0, 0, 1]) # tensor(2)
12 print(t[0, 1, 0]) # tensor(3)
13 print(t[1, 0, 0]) # tensor(5)
14 print(t[1, 1, 1]) # tensor(8)
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
```


Mapping Tensors to Memory

- Tensors are multidimensional arrays
- Elements can be accessed through indices
- Memory / address space is linear — no concept of higher dimensions
- The mapping between memory and tensor indices is determined by its *strides*
- The stride of a dimension specifies how many elements to jump to access the next element along that dimension
- We assume general row-major format in this course — dimensions have descending strides, i.e., the last dimension has stride 1

```
1 import torch
2 # 3D tensor where each dimension size is two
3 t = torch.tensor([[[1, 2],
4                   [3, 4]],
5                  [[5, 6],
6                   [7, 8]]], dtype=torch.int8)
7
8 print(t.shape) # torch.Size([2, 2, 2])
9
10 print(t[0, 0, 0]) # tensor(1)
11 print(t[0, 0, 1]) # tensor(2)
12 print(t[0, 1, 0]) # tensor(3)
13 print(t[1, 0, 0]) # tensor(5)
14 print(t[1, 1, 1]) # tensor(8)
15
16 print(t.untyped_storage())
17 # 1, 2, 3, 4, 5, 6, 7, 8
18 # [torch.storage.UntypedStorage(device=cpu) of
   size 8]
19
20 print(t.stride()) # (4, 2, 1)
21
22
23
24
25
26
27
28
29
```

Mapping Tensors to Memory

- Tensors are multidimensional arrays
- Elements can be accessed through indices
- Memory / address space is linear — no concept of higher dimensions
- The mapping between memory and tensor indices is determined by its *strides*
- The stride of a dimension specifies how many elements to jump to access the next element along that dimension
- We assume general row-major format in this course — dimensions have descending strides, i.e., the last dimension has stride 1

```
1 import torch
2 # 3D tensor where each dimension size is two
3 t = torch.tensor([[[1, 2],
4                   [3, 4]],
5                  [[5, 6],
6                   [7, 8]]], dtype=torch.int8)
7
8 print(t.shape) # torch.Size([2, 2, 2])
9
10 print(t[0, 0, 0]) # tensor(1)
11 print(t[0, 0, 1]) # tensor(2)
12 print(t[0, 1, 0]) # tensor(3)
13 print(t[1, 0, 0]) # tensor(5)
14 print(t[1, 1, 1]) # tensor(8)
15
16 print(t.untyped_storage())
17 # 1, 2, 3, 4, 5, 6, 7, 8
18 # [torch.storage.UntypedStorage(device=cpu) of
   size 8]
19
20 print(t.stride()) # (4, 2, 1)
21
22 mem = bytearray(range(0, 8))
23 t_mem = torch.frombuffer(mem, dtype=torch.int8)
24 print(t_mem) # tensor([1, 2, 3, 4, 5, 6, 7, 8])
25
26 print(t_mem.view(2, 2, 2)) # tensor([[[1, 2],
27                                   #      [3, 4]],
28                                   #      [[5, 6],
29                                   #      [7, 8]]])
```

Mapping Tensors to Memory

- Tensors are multidimensional arrays
- Elements can be accessed through indices
- Memory / address space is linear — no concept of higher dimensions
- The mapping between memory and tensor indices is determined by its *strides*
- The stride of a dimension specifies how many elements to jump to access the next element along that dimension
- We assume general row-major format in this course — dimensions have descending strides, i.e., the last dimension has stride 1

```
1 import torch
2 # 3D tensor where each dimension size is two
3 t = torch.tensor([[[1, 2],
4                   [3, 4]],
5                  [[5, 6],
6                   [7, 8]]], dtype=torch.int8)
7
8 print(t.shape) # torch.Size([2, 2, 2])
9
10 print(t[0, 0, 0]) # tensor(1)
11 print(t[0, 0, 1]) # tensor(2)
12 print(t[0, 1, 0]) # tensor(3)
13 print(t[1, 0, 0]) # tensor(5)
14 print(t[1, 1, 1]) # tensor(8)
15
16 print(t.untyped_storage())
17 # 1, 2, 3, 4, 5, 6, 7, 8
18 # [torch.storage.UntypedStorage(device=cpu) of
   size 8]
19
20 print(t.stride()) # (4, 2, 1)
21
22 mem = bytearray(range(0, 8))
23 t_mem = torch.frombuffer(mem, dtype=torch.int8)
24 print(t_mem) # tensor([1, 2, 3, 4, 5, 6, 7, 8])
25
26 print(t_mem.view(2, 2, 2)) # tensor([[[1, 2],
27                                #         [3, 4]],
28                                #         [[5, 6],
29                                #         [7, 8]]])
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

```
1 import torch
2
3 t = torch.tensor([[[1, 2],
4                   [3, 4]],
5                   [[5, 6],
6                   [7, 8]]], dtype=torch.int8)
7 print(t.shape) # torch.Size([2, 2, 2])
8 print(t.stride()) # (4, 2, 1)
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

```
1 import torch
2
3 t = torch.tensor([[[1, 2],
4                   [3, 4]],
5                   [[5, 6],
6                   [7, 8]]], dtype=torch.int8)
7 print(t.shape) # torch.Size([2, 2, 2])
8 print(t.stride()) # (4, 2, 1)
9
10
11 # view of 3D tensor as 1D tensor
12 # tensor([1, 2, 3, 4, 5, 6, 7, 8])
13 t_1D = t.view(8)
14 print(t_1D.shape) # torch.Size([8])
15 print(t_1D.stride()) # (1,)
16
17
18
19
20
21
22
23
24
25
26
27
28
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

```
1 import torch
2
3 t = torch.tensor([[[1, 2],
4                   [3, 4]],
5                   [[5, 6],
6                   [7, 8]]], dtype=torch.int8)
7 print(t.shape) # torch.Size([2, 2, 2])
8 print(t.stride()) # (4, 2, 1)
9
10
11 # view of 3D tensor as 1D tensor
12 # tensor([1, 2, 3, 4, 5, 6, 7, 8])
13 t_1D = t.view(8)
14 print(t_1D.shape) # torch.Size([8])
15 print(t_1D.stride()) # (1,)
16
17
18 # view of 3D tensor as 2D tensor
19 # tensor([[1, 2, 3, 4],
20 #         [5, 6, 7, 8]])
21 t_2D = t.view(2, 4)
22 print(t_2D.shape) # torch.Size([2, 4])
23 print(t_2D.stride()) # (4, 1)
24
25
26
27
28
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

```
1 import torch
2
3 t = torch.tensor([[[1, 2],
4                   [3, 4]],
5                   [[5, 6],
6                   [7, 8]]], dtype=torch.int8)
7 print(t.shape) # torch.Size([2, 2, 2])
8 print(t.stride()) # (4, 2, 1)
9
10
11 # view of 3D tensor as 1D tensor
12 # tensor([1, 2, 3, 4, 5, 6, 7, 8])
13 t_1D = t.view(8)
14 print(t_1D.shape) # torch.Size([8])
15 print(t_1D.stride()) # (1,)
16
17
18 # view of 3D tensor as 2D tensor
19 # tensor([[1, 2, 3, 4],
20 #         [5, 6, 7, 8]])
21 t_2D = t.view(2, 4)
22 print(t_2D.shape) # torch.Size([2, 4])
23 print(t_2D.stride()) # (4, 1)
24
25
26 # all three tensors share the same memory
27 print(t.data_ptr() == t_1D.data_ptr() ==
28       t_2D.data_ptr())
29 # True
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

Example

- What is the shape of `t_mystery = t.view(4, 2)`?
- What are the strides of `t_mystery`?
- What is the output of `print(t_mystery)`?

```
1 import torch
2
3 t = torch.tensor([[[[1, 2],
4                     [3, 4]],
5                     [[5, 6],
6                     [7, 8]]], dtype=torch.int8)
7 print(t.shape) # torch.Size([2, 2, 2])
8 print(t.stride()) # (4, 2, 1)
9
10
11 # view of 3D tensor as 1D tensor
12 # tensor([1, 2, 3, 4, 5, 6, 7, 8])
13 t_1D = t.view(8)
14 print(t_1D.shape) # torch.Size([8])
15 print(t_1D.stride()) # (1,)
16
17
18 # view of 3D tensor as 2D tensor
19 # tensor([[1, 2, 3, 4],
20 #         [5, 6, 7, 8]])
21 t_2D = t.view(2, 4)
22 print(t_2D.shape) # torch.Size([2, 4])
23 print(t_2D.stride()) # (4, 1)
24
25
26 # all three tensors share the same memory
27 print(t.data_ptr() == t_1D.data_ptr() ==
28       t_2D.data_ptr())
29 # True
```


Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

Example

- What is the shape of `t_mystery = t.view(4, 2)`?
`torch.Size([4, 2])`
- What are the strides of `t_mystery`?
- What is the output of `print(t_mystery)`?

```
1 import torch
2
3 t = torch.tensor([[[[1, 2],
4                     [3, 4]],
5                     [[5, 6],
6                     [7, 8]]], dtype=torch.int8)
7 print(t.shape) # torch.Size([2, 2, 2])
8 print(t.stride()) # (4, 2, 1)
9
10
11 # view of 3D tensor as 1D tensor
12 # tensor([1, 2, 3, 4, 5, 6, 7, 8])
13 t_1D = t.view(8)
14 print(t_1D.shape) # torch.Size([8])
15 print(t_1D.stride()) # (1,)
16
17
18 # view of 3D tensor as 2D tensor
19 # tensor([[1, 2, 3, 4],
20 #         [5, 6, 7, 8]])
21 t_2D = t.view(2, 4)
22 print(t_2D.shape) # torch.Size([2, 4])
23 print(t_2D.stride()) # (4, 1)
24
25
26 # all three tensors share the same memory
27 print(t.data_ptr() == t_1D.data_ptr() ==
28       t_2D.data_ptr())
28 # True
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

Example

- What is the shape of `t_mystery = t.view(4, 2)`?
`torch.Size([4, 2])`
- What are the strides of `t_mystery`?
`(2, 1)`
- What is the output of `print(t_mystery)`?

```
1 import torch
2
3 t = torch.tensor([[[1, 2],
4                   [3, 4]],
5                   [[5, 6],
6                   [7, 8]]], dtype=torch.int8)
7 print(t.shape) # torch.Size([2, 2, 2])
8 print(t.stride()) # (4, 2, 1)
9
10
11 # view of 3D tensor as 1D tensor
12 # tensor([1, 2, 3, 4, 5, 6, 7, 8])
13 t_1D = t.view(8)
14 print(t_1D.shape) # torch.Size([8])
15 print(t_1D.stride()) # (1,)
16
17
18 # view of 3D tensor as 2D tensor
19 # tensor([[1, 2, 3, 4],
20 #         [5, 6, 7, 8]])
21 t_2D = t.view(2, 4)
22 print(t_2D.shape) # torch.Size([2, 4])
23 print(t_2D.stride()) # (4, 1)
24
25
26 # all three tensors share the same memory
27 print(t.data_ptr() == t_1D.data_ptr() ==
28       t_2D.data_ptr())
28 # True
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

Example

- What is the shape of `t_mystery = t.view(4, 2)`?
`torch.Size([4, 2])`
- What are the strides of `t_mystery`?
`(2, 1)`
- What is the output of `print(t_mystery)`?
`tensor([[1, 2], [3, 4], [5, 6], [7, 8]])`

```
1 import torch
2
3 t = torch.tensor([[[1, 2],
4                   [3, 4]],
5                   [[5, 6],
6                   [7, 8]]], dtype=torch.int8)
7 print(t.shape) # torch.Size([2, 2, 2])
8 print(t.stride()) # (4, 2, 1)
9
10
11 # view of 3D tensor as 1D tensor
12 # tensor([1, 2, 3, 4, 5, 6, 7, 8])
13 t_1D = t.view(8)
14 print(t_1D.shape) # torch.Size([8])
15 print(t_1D.stride()) # (1,)
16
17
18 # view of 3D tensor as 2D tensor
19 # tensor([[1, 2, 3, 4],
20 #         [5, 6, 7, 8]])
21 t_2D = t.view(2, 4)
22 print(t_2D.shape) # torch.Size([2, 4])
23 print(t_2D.stride()) # (4, 1)
24
25
26 # all three tensors share the same memory
27 print(t.data_ptr() == t_1D.data_ptr() ==
28       t_2D.data_ptr())
28 # True
```

Tensor Slicing and Striding

- Slicing and striding can be used to create views of tensors without copying memory
- Syntax: `tensor[start:stop:step, ...]`
 - `start` (default: 0) is the index to start from (inclusive)
 - `stop` (default: size of dimension) is the index to stop at (exclusive)
 - `step` (default: 1) is the step size to take between indices
 - `:` includes the whole dimension

```
1 import torch
2
3 t = torch.tensor([[[[1, 2],
4                     [3, 4]],
5                     [[5, 6],
6                     [7, 8]]], dtype=torch.int8)
7
8 # tensor([[1, 2, 3, 4],
9 #         [5, 6, 7, 8]])
10 t_2D = t.view(2, 4)
11
12 # tensor([[1, 3],
13 #         [5, 7]])
14 t_sliced = t_2D[0:2:1, 0:4:2]
15 t_sliced = t_2D[:, ::2] # same as above
16
17 # no unit stride, i.e., no stride-1 dimension
18 print(t_sliced.stride()) # (4, 2)
19 print(t_sliced.is_contiguous()) # False
20
21
22 # slice 3D tensor into two 2D tensors
23 m0 = t[0, :, :] # tensor([[1, 2],
24                       #         [3, 4]])
25
26 m1 = t[1, :, :] # tensor([[5, 6],
27                       #         [7, 8]])
```

Example

```
print(t[0, 0, :])
```

```
# tensor([1, 2])
```

Einsum

Einsum

- An *einsum* describes multidimensional summation over index products
- Example: matrix multiplication ($C = A \cdot B$):
 - $c_{ik} = \sum_j a_{ij} \cdot b_{jk}$
 - `einsum(ij, jk -> ik, A, B)`
- *einsum* syntax:
 - Each dimension is represented by a label, e.g., `i`, `j`, `k`
 - Tensors are represented by their dimension labels, e.g., `ij` for a 2D tensor
 - Input tensors are comma-separated, e.g., `ij, jk`
 - Labels of the output tensor are specified after `->`, e.g., `ij, jk -> ik`
 - For shared dimensions, the same label is used, e.g., a matrix multiplication has three unique dimensions
 - Input elements along corresponding dimensions are multiplied, e.g., $a_{ij} * b_{jk}$
 - If a dimension label only appears in the input tensors, it is summed over, e.g., $\sum_j$

Einsum Examples

Example

$$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$$

```
C = torch.einsum('abjk, cjk -> abc', A, B)
```

Einsum Examples

Example

$$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$$

```
C = torch.einsum('abjk, cjk -> abc', A, B)
```

$$c_{bac} = \sum_{jk} a_{jbka} \cdot b_{kcj}$$

Einsum Examples

Example

$$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$$

```
C = torch.einsum('abjk, cjk -> abc', A, B)
```

$$c_{bac} = \sum_{jk} a_{jbka} \cdot b_{kcj}$$

```
C = torch.einsum('jbka, kcj -> bac', A, B)
```

Einsum Examples

Example

$$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$$

```
C = torch.einsum('abjk, cjk -> abc', A, B)
```

$$c_{bac} = \sum_{jk} a_{jbka} \cdot b_{kcj}$$

```
C = torch.einsum('jbka, kcj -> bac', A, B)
```

$$d_{xy} = \sum_k a_{xk} \cdot b_{yk} \cdot c_{xy}$$

Einsum Examples

Example

$$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$$

```
C = torch.einsum(abjk, cjk -> abc, A, B)
```

$$c_{bac} = \sum_{jk} a_{jbka} \cdot b_{kcj}$$

```
C = torch.einsum(jbka, kcj -> bac, A, B)
```

$$d_{xy} = \sum_k a_{xk} \cdot b_{yk} \cdot c_{xy}$$

```
D = torch.einsum(xk, yk, xy -> xy, A, B, C)
```

Einsum Examples

Example

$$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$$

```
C = torch.einsum(abjk, cjk -> abc, A, B)
```

$$c_{bac} = \sum_{jk} a_{jbka} \cdot b_{kcj}$$

```
C = torch.einsum(jbka, kcj -> bac, A, B)
```

$$d_{xy} = \sum_k a_{xk} \cdot b_{yk} \cdot c_{xy}$$

```
D = torch.einsum(xk, yk, xy -> xy, A, B, C)
```

```
D = torch.einsum(xk, yk, zk -> xyz, A, B, C)
```

Einsum Examples

Example

$$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$$

```
C = torch.einsum(abjk, cjk -> abc, A, B)
```

$$c_{bac} = \sum_{jk} a_{jbka} \cdot b_{kcj}$$

```
C = torch.einsum(jbka, kcj -> bac, A, B)
```

$$d_{xy} = \sum_k a_{xk} \cdot b_{yk} \cdot c_{xy}$$

```
D = torch.einsum(xk, yk, xy -> xy, A, B, C)
```

$$d_{xyz} = \sum_k a_{xk} \cdot b_{yk} \cdot c_{zk}$$

```
D = torch.einsum(xk, yk, zk -> xyz, A, B, C)
```

Einsum Examples

Operation	Mathematical	Summation	<i>einsum</i>
Matrix product	$A \cdot B$	$c_{pr} = \sum_q a_{pq} b_{qr}$	<code>#('pq,qr->pr', A, B)</code>
Transposed matrix product	$A^T \cdot B$	$c_{pr} = \sum_q a_{qp} b_{qr}$	<code>#('qp,qr->pr', A, B)</code>
Hadamard product	$A \odot B$	$c_{pq} = a_{pq} b_{pq}$	<code>#('pq,pq->pq', A, B)</code>
sq'd Frobenius norm	$\ A\ _F^2$	$c = \sum_p \sum_q a_{pq} a_{pq}$	<code>#('pq,pq->', A, A)</code>
Khatri-Rao product	$A \odot B$	$c_{pqr} = a_{pq} b_{qr}$	<code>#('pq,qr->pqr', A, B)</code>
Kronecker product	$A \otimes B$	$c_{pqrs} = a_{pq} b_{rs}$	<code>#('pq,rs->pqrs', A, B)</code>